Magpie — Sprint 3 Technical Spec (Phase 2 cont.: Commodity-Live Valuation + Gold Schema)

Audience: Codex + Regan. **Builds on** Sprint 0–2. **Status: planning / review — implement only after this spec is reviewed and approved.**

> Build only the locked Sprint 3 scope (D-31…D-38). Broader per-category schemas, catalogue research links, listing, eBay, OCR/AI, realized-profit dashboards, and any trading/buying logic are **out of scope** (Sprint 4 or later phases).

Inherited from Sprint 0–2 (not restated): `TimeStampedUUIDModel`; `apps/` + `integrations/` ports; DRF defaults; **Decimal** for all money/weight; `manage.py backup` includes new tables; the **real-PostgreSQL CI gate** (D-22). The `ValuationReport`, `FeeCalculator`, `FeeSchedule`, `Comparable`, `ResearchRecord`, and the strategy enum (`comp_based` / `commodity_manual` / `commodity_live`) already exist from Sprint 2 — Sprint 3 **fills in** **`commodity_live`** and adds the metals adapter, a spot cache, and the gold schema. No new Django app.

---

0. Locked decisions this spec implements

- **D-31** Sprint 3 = `MetalsPriceAdapter` + `commodity_live` + first gold/precious-metals schema; broader schemas + catalogue links → Sprint 4.
- **D-32** Adapter behind the port; **provider not locked** (build-time choice); **env-var config**; no vendor SDK/HTTP outside `integrations/metals.py`; **fake adapter in CI**; works with no provider configured.
- **D-33 DB-backed** spot cache (metal+currency+provider, ~1h TTL, manual refresh).
- **D-34** `commodity_live` = `weight_g × fineness × spot_per_g` (Decimal), normalized to **AUD/gram**; full provenance snapshot into the report.
- **D-35 Graceful fallback** to `commodity_manual`; show source + timestamp; never depend on the feed.
- **D-36** Gold schema with **partial capture**; weight + fineness/karat required only for a commodity valuation.
- **D-37** Override always wins; **not a trading / financial-advice tool**.

- **D-38** Out-of-scope list (see §9).

---

1. What Sprint 3 adds / changes

File	Change
`integrations/metals.py`	Implement `HttpMetalsPriceAdapter` (concrete) + `FakeMetalsPriceAdapter`; keep the ABC; add `get_metals_adapter()` factory + `MetalsUnavailable` exception.
`apps/valuation/models.py`	Add `MetalSpotCache`.
`apps/valuation/strategies.py`	Implement `CommodityLiveStrategy` (currently raises); add `karat_to_fineness` + per-oz→per-g helpers.
`apps/valuation/services.py`	`get_spot(...)` (cache-aware) + the `commodity_live` build path (provenance snapshot + fallback).
`apps/catalog/profiles.py`	Add `PreciousMetalsSchema`; register for the gold category `profile_key`.
`apps/valuation/views.py` + `apps/api`	Metals spot endpoint (+ refresh); wire `commodity_live` into the existing valuation-report create.
`config/settings/*` + `.env.example`	`METALS_*` env vars.
`frontend/`	Cockpit live-spot controls + gold item fields.

---

2. Models / code

2.1 `MetalSpotCache` (`apps/valuation/models.py`)

```
class Metal(models.TextChoices):
    GOLD = "gold", "Gold"; SILVER = "silver", "Silver"
    PLATINUM = "platinum", "Platinum"; PALLADIUM = "palladium", "Palladium"
```

```
class MetalSpotCache(TimeStampedUUIDModel):
    metal = models.CharField(max_length=12, choices=Metal.choices)
    currency = models.CharField(max_length=3, default="AUD")
    provider = models.CharField(max_length=60)
    price_per_gram = models.DecimalField(max_digits=14, decimal_places=6)   # nor
    provider_price = models.DecimalField(max_digits=14, decimal_places=6)   # as
    provider_units = models.CharField(max_length=20)                        # e.g
    as_of = models.DateTimeField()       # provider's quote time
    fetched_at = models.DateTimeField()  # when we fetched

    class Meta:
        constraints = [models.UniqueConstraint(fields=["metal", "currency", "prov
                                               name="uniq_metal_currency_provider
```

TTL = `settings.METALS_CACHE_TTL_SECONDS` **(default 3600); a row is stale if** `fetched_at` **is older than the TTL.**

2.2 Adapter (`integrations/metals.py`)

```
@dataclass
class SpotQuote:
    metal: str; currency: str
    price_per_gram: Decimal; provider_price: Decimal; provider_units: str
    source: str; as_of: datetime; fetched_at: datetime; cache_hit: bool

class MetalsUnavailable(Exception): ...

class MetalsPriceAdapter(ABC):  # existing port
    @abstractmethod
    def spot_price(self, metal: str, currency: str = "AUD") -> SpotQuote: ...

class HttpMetalsPriceAdapter(MetalsPriceAdapter):
    # Concrete. Provider + key from settings (env). Calls the provider, normalize
    # (per-troy-oz -> /Decimal('31.1034768')), returns SpotQuote(cache_hit=False)
    # Raises MetalsUnavailable on misconfig / network / parse failure.
    # The ONLY place a metals vendor SDK/HTTP call may appear.

class FakeMetalsPriceAdapter(MetalsPriceAdapter):
    # Deterministic fixed quote for tests/CI. No network.

def get_metals_adapter() -> MetalsPriceAdapter:
    # Returns HttpMetalsPriceAdapter if settings.METALS_PROVIDER is set,
    # else an Unconfigured adapter whose spot_price raises MetalsUnavailable.
```

Provider selection happens **at build time after checking current API terms** — not encoded as a locked decision. Prefer a provider that can return an AUD-denominated precious-metal quote (avoids a separate FX hop); otherwise take the provider's units and normalize. Keys come from env only.

2.3 Spot service (`apps/valuation/services.py`)

```
def get_spot(metal, currency="AUD", *, force_refresh=False) -> SpotQuote:
    # 1. If a fresh MetalSpotCache row exists (within TTL) and not force_refresh
    # 2. Else adapter.spot_price(metal, currency); upsert the cache row; return i
    # 3. If the adapter raises MetalsUnavailable and no usable cache row exists -
```

2.4 `CommodityLiveStrategy` (`apps/valuation/strategies.py`)

```
def karat_to_fineness(karat) -> Decimal:
    return (Decimal(str(karat)) / Decimal(24))   # 24k=1.0, 18k=0.75, 9k=0.375

class CommodityLiveStrategy:  # key = "commodity_live"
    def estimate(self, *, item, included_comps, inputs):
        metal = inputs["metal"]
        weight = Decimal(str(inputs["weight_g"]))
        fineness = Decimal(str(inputs["fineness"])) if inputs.get("fineness") \
                   else karat_to_fineness(inputs["karat"])
        quote = get_spot(metal, inputs.get("currency", "AUD"))   # may raise Meta
        intrinsic = (weight * fineness * quote.price_per_gram)
        margin = Decimal(str(inputs.get("buy_margin_pct", 0)))
        # suggested = patient = intrinsic; fast_sale = intrinsic * (1 - margin/10
        # low = median = high = intrinsic (single point)
        # (caller snapshots provenance into report.inputs — see 2.5)
```

`commodity_live` was registered-but-raising in Sprint 2; this implements it. All maths is `Decimal`.

2.5 Provenance snapshot (into `ValuationReport.inputs`)

On a `commodity_live` report, store: `metal` , `currency` , `normalized_price_per_g` , `provider_price` , `provider_units` , `source` , `as_of` , `fetched_at` , `cache_hit` , `weight_g` , `fineness` , `calculated_intrinsic_value` . The report must stay reproducible years later even if the provider disappears (mirrors how Sprint 2 pinned the `FeeSchedule`).

2.6 `PreciousMetalsSchema` (`apps/catalog/profiles.py`)

```
class PreciousMetalsSchema:        # profile_key = "gold" (also used for other pr
    profile_key = "gold"
    ALLOWED = {"metal", "weight_g", "fineness", "karat", "form", "notes"}
    FORMS = {"specimen", "jewellery", "coin", "scrap", "nugget", "other"}
    def validate(self, attributes: dict) -> dict:
        # PARTIAL CAPTURE: nothing required to SAVE the item. metal defaults to "
        # Reject invalid values WHEN PRESENT:
        #   weight_g must be > 0; fineness must be in (0, 1]; karat must be 1..24
        # Unknown keys: ignore or reject (choose reject for tidiness).
        ...
```

Register via `register(PreciousMetalsSchema())` and set the gold `ProductCategory.profile_key = "gold"`. The commodity-valuation requirement — `weight_g` plus `fineness` or `karat` — is enforced at valuation-report create, *not* in this schema, so item capture stays partial (D-36).

---

3. API

Method	Path	Purpose
GET	`/api/metals/spot/?metal=gold&currency=AUD[&refresh=true]`	Returns a `SpotQuote` (price_per_gram, provider_price, provider_units, source, as_of, fetched_at, cache_hit). `refresh=true` forces a re-fetch. **503 + clear message** if the provider is unconfigured/unavailable and there's no cache.
POST	`/api/items/{id}/valuation-reports/` (existing)	With `strategy=commodity_live` + `inputs` (`metal`, `weight_g`, `fineness` | `karat`, `buy_margin_pct`): backend fetches spot, snapshots provenance, computes intrinsic. **On `MetalsUnavailable`**, respond with `{"needs_manual_spot":`

		`true}` (no live price) so the UI prompts manual entry → `commodity_manual`. **422** if `weight_g` or `fineness` / `karat` is missing for a commodity report.

Override behaviour is unchanged from Sprint 2 (any price editable with an `override_reason`).

---

4. Frontend (cockpit additions)

- **Item form (gold/precious-metals):** `metal`, `weight_g`, `fineness`, `karat`, `form` — all optional at capture (partial capture).
- **Valuation panel, commodity path:** a **"Fetch live spot"** button → shows price/gram with **source + `as_of` timestamp + cache age**; a **Refresh** button (`refresh=true`); the computed intrinsic value. If the feed is unconfigured/unavailable, a **clear message + manual spot entry** (the `commodity_manual` path). Source + timestamp are always shown next to a live price; the manual **override (with reason) is always available**.

---

5. Config (`.env.example` additions)

```
METALS_PROVIDER=            # blank = disabled (commodity_live falls back to manu
METALS_API_KEY=
METALS_BASE_CURRENCY=AUD
METALS_CACHE_TTL_SECONDS=3600
```

The app runs fine with `METALS_PROVIDER` blank — `commodity_live` is simply disabled and the cockpit uses manual spot entry. **Keys live in env only; never commit them.**

---

6. Acceptance criteria

1. Migrations apply on real PostgreSQL in CI (`MetalSpotCache` added); **Postgres CI gate green**.

2. **CI uses `FakeMetalsPriceAdapter` — no live network call.** Strategy behaviour is verified deterministically.
3. `commodity_live` computes intrinsic (`Decimal`) = `weight × fineness × AUD/g spot`; a `karat` input converts to fineness; per-troy-oz provider prices convert with `31.1034768`.
4. Spot cache: a row fresh within TTL returns `cache_hit=true`; `refresh=true` re-fetches; keyed per metal+currency+provider; **survives restart** (DB-backed).
5. **Graceful degradation:** with `METALS_PROVIDER` unset (or a failing fetch and no cache), the cockpit shows a clear message and allows manual spot entry (`commodity_manual`); the app stays fully usable.
6. Provenance snapshot (all §2.5 fields) is stored on the report; the report is reproducible without the provider.
7. Gold schema: an item saves with partial fields; invalid values are rejected **when present**; `weight_g` + `fineness` / `karat` are required only at commodity-valuation create (422 otherwise).
8. **Source + timestamp shown** next to any live price; user **override (with reason) still wins**.
9. `manage.py backup` includes `MetalSpotCache`; restore on an empty DB passes. Backend + frontend tests pass.

---

7. Test plan

Backend (`pytest` + `pytest-django`, `FakeMetalsPriceAdapter` injected):

- *Strategy/maths:* intrinsic value; `karat_to_fineness` (24/18/14/9k); per-oz→per-g; **Decimal not float**.
- *Cache:* TTL freshness; `force_refresh`; per-key uniqueness.
- *Fallback:* provider unset / adapter raises → report create returns `needs_manual_spot`; manual spot path computes (`commodity_manual`).
- *Schema:* partial capture saves; negative weight / fineness > 1 / karat > 24 rejected; commodity report missing weight → 422.
- *API:* spot endpoint (cached + `refresh` + unconfigured 503); commodity_live report create (live + fallback); override with reason.

- *Provenance:* snapshot fields present + correct.
- *Backup:* `MetalSpotCache` present after backup→restore.

Frontend (Vitest + RTL — light): live-spot panel shows source/timestamp; the manual-fallback UI appears when unconfigured; gold fields are optional on the form.

CI: Postgres gate, **fake adapter only — no external metals call**.

---

8. Seed data (extend `seed.py`, idempotent)

A gold sample item with `attributes {metal: "gold", weight_g, fineness (or karat), form}`; optionally one `MetalSpotCache` row (`provider="seed-fake"`) and a `commodity_live ValuationReport` carrying a full provenance snapshot — all clearly labelled as seed/fake data (no real provider call).

---

9. Out of scope for Sprint 3 (restated — D-38)

Broader per-category schemas (stamps/coins/phones) · catalogue research links (Numista/Colnect/Stanley Gibbons/Renniks) · listing generation · eBay write/publish · eBay Browse · OCR/AI · `SaleRecord` /realized-profit dashboards · automated trading/buying logic.

Hand-off notes for the implementing agent

- **Choose the metals provider at build time** after checking its current API terms and free-tier limits. Put the key in env; **never commit it**. Keep all provider HTTP/SDK code inside `integrations/metals.py` — nothing leaks into `valuation` / `catalog`.
- **CI must never hit the network** — inject `FakeMetalsPriceAdapter`.
- The app must work with **no provider configured** (commodity_live → manual). Never silently trust a live price: show source + timestamp, and the user override always wins.
- This is **valuation support for inventory — not trading, financial advice, or automated buying.**
- When ambiguous, prefer the smaller implementation that satisfies §6.